

Today's menu

Control structures (loops/ conditional executions)

if, else

for

while

repeat

break

next

switch



Conditional executions control the flow of the execution.



Draw a star on star-shaped cookies and draw a heart on heart-shaped cookies.



if-else

Draw a star on star-shaped cookies and draw a heart on heart-shaped cookies.

if-else

```
if (condition) {  
  # do something  
} else {  
  # do something else  
}
```

Example

```
test_even_odd <- function(x){  
  if (x %% 2 == 0){  
    print("even number")  
  } else {  
    print("odd number")  
  }  
}
```

```
test_even_odd(5)
```

```
[1] "odd number"
```

```
test_even_odd(6)
```

```
[1] "even number"
```

ifelse: vectorization with ifelse

```
ifelse(condition, TRUE condition output, FALSE condition output)
```

Example

```
test_even_odd_v2 <- function(x){  
  ifelse(x %% 2 == 0, "even number", "odd number")  
}
```

```
test_even_odd_v2(5)
```

```
FALSE [1] "odd number"
```

```
test_even_odd_v2(c(1,6))
```

```
FALSE [1] "odd number" "even number"
```

Difference between if, else and ifelse

if, else

```
test_even_odd <- function(x){  
  if (x %% 2 == 0) {  
    print("even number")  
  } else {  
    print("odd number")  
  }  
}
```

```
test_even_odd(5)
```

```
FALSE [1] "odd number"
```

```
test_even_odd(c(1,6))  
# returns an error
```

ifelse

```
test_even_odd_v2 <- function(x){  
  ifelse (x %% 2 == 0, "even number", "odd number")  
}  
  
test_even_odd_v2(5)
```

```
FALSE [1] "odd number"
```

```
test_even_odd_v2(c(1,6))
```

```
FALSE [1] "odd number" "even number"
```

Nested if-else

- Multiple conditions

```
grade_marks <- function(marks){  
  if (marks < 20) {  
    "D"  
  } else if (marks <= 50) {  
    "C"  
  } else if (marks <= 60) {  
    "B"  
  } else {  
    "A"  
  }  
}  
  
grade_marks(75)
```

[1] "A"

Your turn

R for Data Science-Exercises 19.4.4 - Q2

[R for Data Science](#)

Table of contents

- [Welcome](#)
- [1 Introduction](#)
- Explore
- [2 Introduction](#)
- [3 Data visualisation](#)
- [4 Workflow: basics](#)
- [5 Data transformation](#)
- [6 Workflow: scripts](#)
- [7 Exploratory Data Analysis](#)

19 Functions

You're reading the first edition of R4DS; for the latest on this topic see the [Functions chapter](#) in the second edition.

19.1 Introduction

One of the best ways to improve your reach as a data scientist is to write functions. Functions allow you to automate common tasks in a more powerful and general way than copy-and-pasting. Writing a function has three big advantages over using copy-and-paste:

On this page

- [19 Functions](#)
- [19.1 Introduction](#)
- [19.2 When should you write a function?](#)
- [19.3 Functions are for humans and computers](#)
- [19.4 Conditional execution](#)
- [19.5 Function arguments](#)
- [19.6 Return values](#)

Help:

`lubridate::now()` and `lubridate::hour()`

10:00

if

```
check.negative <- function(x){  
  if (x < 0 ) {  
    print("X is negative")  
  }  
}
```

```
check.negative(-10)
```

```
[1] "X is negative"
```

```
check.negative(10)
```




For the first 50 cookies, fill the center of cookies with jam.



for

For the first 50 cookies, fill the center of cookies with jam.

for loop

- execute a block of code a specific number of times or until the end of a sequence.

```
for (i in 1:5) {  
  print(i*100)  
}
```

```
[1] 100  
[1] 200  
[1] 300  
[1] 400  
[1] 500
```

```
continents <- c("Asia", "EU", "AUS", "NA", "SA", "Africa")
```

```
for (i in 1:4) {  
  print(continents[i])  
}
```

```
[1] "Asia"  
[1] "EU"  
[1] "AUS"  
[1] "NA"
```

```
for (i in 1:4) print(continents[i])
```

```
[1] "Asia"  
[1] "EU"  
[1] "AUS"  
[1] "NA"
```

```
for (i in seq(continents)) {  
  print(continents[i])  
}
```

```
[1] "Asia"  
[1] "EU"  
[1] "AUS"  
[1] "NA"  
[1] "SA"  
[1] "Africa"
```

Your turn

Write a for loop that iterates over the numbers 1 to 4 and prints the squared of each number using `print()`.

03:00



For the first 50 cookies, draw a star on star-shaped cookies and draw a heart on heart-shaped cookies.



nested loops

For the first 50 cookies, draw a star on star-shaped cookies and draw a heart on heart-shaped cookies.

Nested loops

```
a <- 1:10

testfun <- function(x){
  for (x in 1:5) {
    if (x %% 2 == 0) {
      print("even number")
    } else {
      print("odd number")
    }
  }
}

testfun(a)
```

```
[1] "odd number"
[1] "even number"
[1] "odd number"
[1] "even number"
[1] "odd number"
```

Nested loops

```
mat <- matrix(1:6, ncol=2)
mat
```

```
      [,1] [,2]
[1,]     1     4
[2,]     2     5
[3,]     3     6
```

```
for (i in 1:3) {
  for (j in 1:2) {
    print(mat[i, j])
  }
}
```

```
[1] 1
[1] 4
[1] 2
[1] 5
[1] 3
[1] 6
```

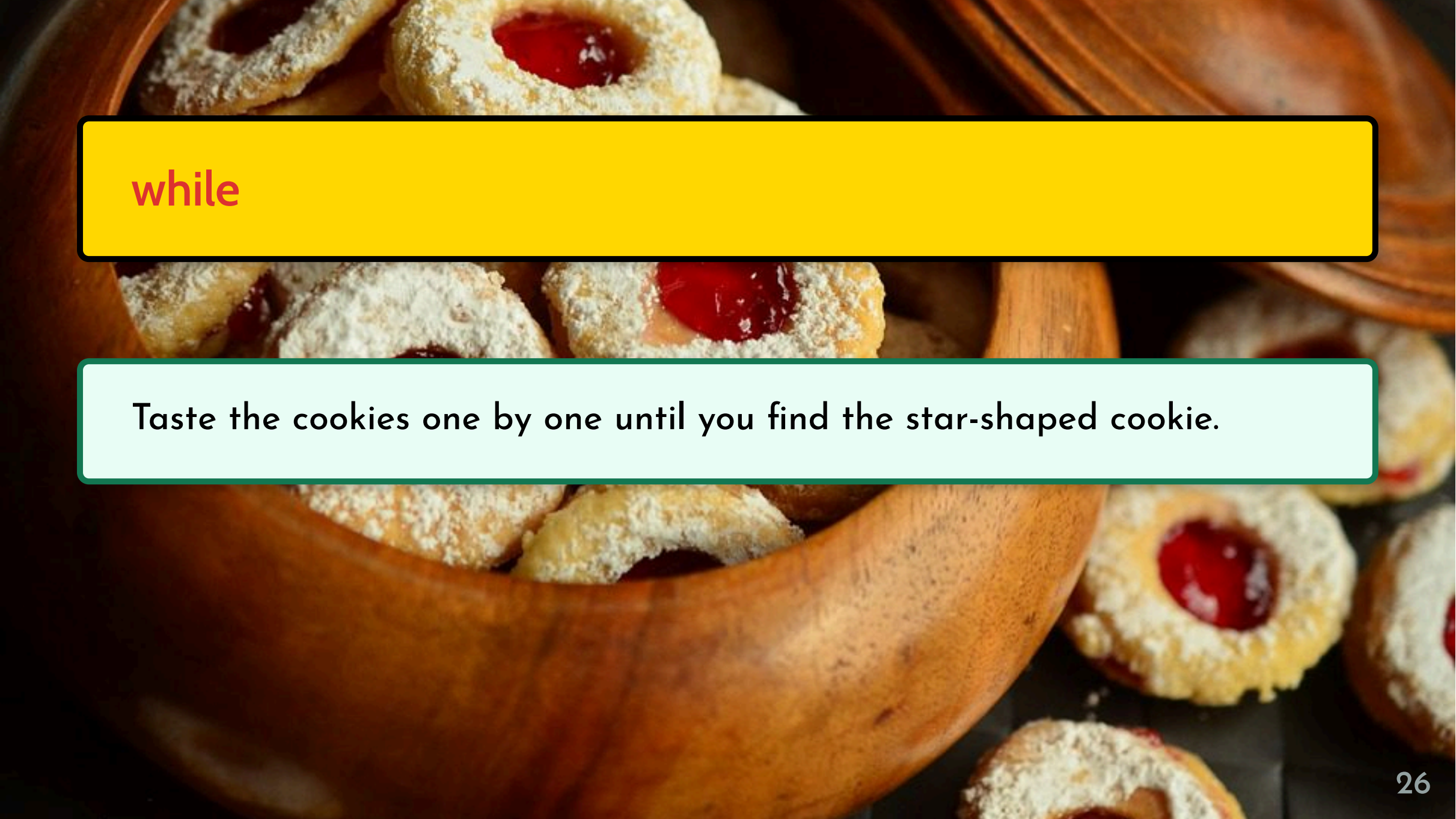
Your turn

Write a function to count the number of even numbers in a vector.

08:00



Taste the cookies one by one until you find the star-shaped cookie.

A close-up photograph of a wooden bowl filled with round, golden-brown cookies. Each cookie has a hole in the center and is filled with a vibrant red jam. The cookies are dusted with white powdered sugar. The bowl is made of light-colored wood and sits on a dark surface. A wooden spoon is partially visible in the background.

while

Taste the cookies one by one until you find the star-shaped cookie.

while

```
i <- 1 # initial value

while (i != 10) {

  print(i)
  i <- i + 1 # increment

}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
```

```
i <- 1 # initial value

while (i < 10) {

  print(i)
  i <- i + 1 # increment

}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
```

Your turn

Can you guess the output?

```
i <- 1 # initial value
while (i == 10) {
  print(i)
  i <- i + 1 # increment
}
```

01:00

Your turn

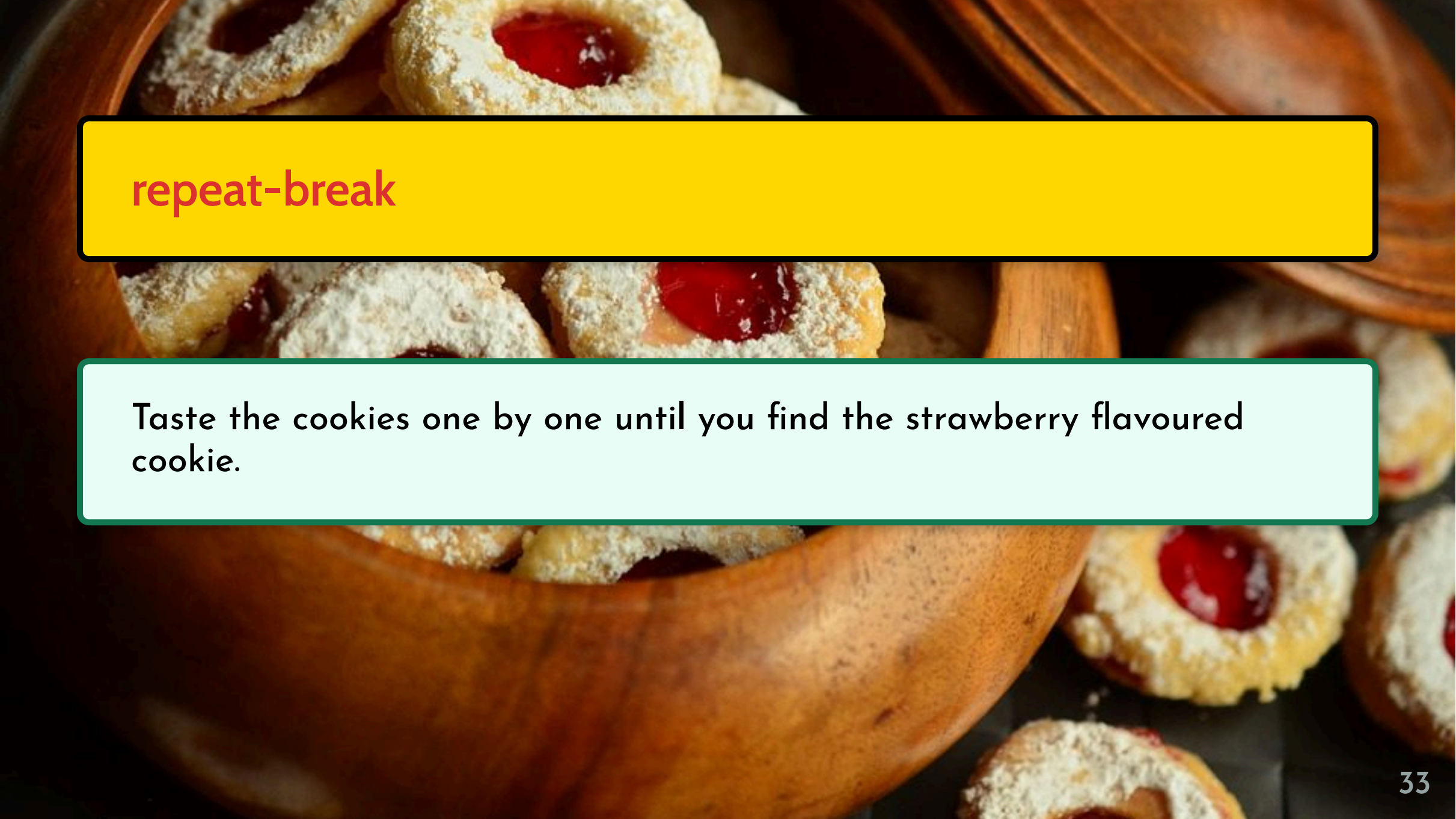
Generate the first **n** numbers of the Fibonacci Sequence using while loop.

0 1 1 2 3 5 ...

04:00



Taste the cookies one by one until you find the strawberry flavoured cookie.



repeat-break

Taste the cookies one by one until you find the strawberry flavoured cookie.

repeat and break

- Iterate over a block of code multiple number of times.
- No condition check in repeat loop to exit the loop.
- The only way to exit a repeat loop is to call break.

repeat and break

Example 1

```
x <- 5

repeat {
  print(x)
  x = x+1
  if (x == 10){
    break
  }
}
```

```
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
```

Example 2

```
set.seed(1)

repeat {
  x<-runif(1, 5, 10)
  print(x)
  if(x < 6.1){
    break
  }
}
```

```
[1] 6.327543
[1] 6.860619
[1] 7.864267
[1] 9.541039
[1] 6.00841
```

Your turn

1. Write a function to print random numbers from the standard normal distribution but stops (breaks) if you get a number bigger than 1.
2. Using next adapt the loop from last exercise so that doesn't print negative numbers.



Skip cracked muffins and decorate the rest.



next

Skip cracked muffins and decorate the rest.

next

```
for(i in 1:10) {  
  if(i <= 5) {  
    next # Skip the first 5 iterations  
  }  
  print(i)  
}
```

```
[1] 6  
[1] 7  
[1] 8  
[1] 9  
[1] 10
```




Green base - roses, Red - small dots, Pink - stars, ...

A close-up photograph of a tray filled with cupcakes. Each cupcake is topped with a swirl of white frosting piped into a rose shape. The cupcakes are arranged in rows, and some have green liners while others have red liners. The frosting is thick and creates a textured, spiral pattern.

switch

Green base - roses, Red - small dots, etc..

switch

When you want a function to do different things in different circumstances, then the switch function can be useful.

```
feelings <- c("sad", "afraid")

for (i in feelings){
  print(
    switch(i,
      happy = "I am glad you are happy",
      afraid = "There is nothing to fear",
      sad = "Cheer up",
      angry = "Calm down now"
    )
  )
}
```

```
[1] "Cheer up"
```

```
[1] "There is nothing to fear"
```

Single element - working

```
x <- "a"
v <- switch(x, "a" = "apple",
             "b" = "banana",
             "c" = "cherry")

v
```

[1] "apple"

Multiple elements - not working

```
switch(c("a", "b"), "a" = "apple", # not v
      "b" = "banana",
      "c" = "cherry")
```

Multiple element - use for loop

```
vect <- c("a", "b")
for(i in seq(vect)){
  print(switch(i, "a" = "apple",
              "b" = "banana",
              "c" = "cherry"))
}
```

```
## [1] "apple"
## [1] "banana"
```